

Sakura Saki Final Project Documentation

Project Title

Sakura Saki - Beauty Salon Management System

Project Overview

Sakura Saki is a web-based beauty salon management system designed to digitalize the main operations of a salon. The system combines customer account handling, service and package management, staff management, appointment scheduling, admin reporting, and post-visit reviews into one integrated platform. This project is developed as an Object-Oriented Programming assignment using Java web technologies, with a focus on modular development, CRUD functionality, and clear application of OOP principles.

This documentation reflects the current proposed workload distribution across six members. Since the module list is still not fully finalized, this document should be treated as the latest agreed project draft for implementation, reporting, and viva preparation.

Project Objectives

- Build a web-based Java application that allows users to manage and interact with salon services and records.
- Apply Object-Oriented Programming concepts such as encapsulation, inheritance, abstraction, and polymorphism.
- Implement at least three CRUD operations in each module where applicable.
- Provide user-friendly interfaces for different system roles such as customer, staff, and admin.
- Use proper backend structure with entities, repositories, services, controllers, and UI templates.
- Support either file handling or MySQL database persistence depending on the final implementation setup.

Suggested Technology Stack

- **Backend:** Java, Spring Boot, Spring MVC, Spring Security, Spring Data JPA
- **Frontend:** Thymeleaf, HTML, CSS, Bootstrap, JavaScript
- **Database:** MySQL with JPA/Hibernate ORM
- **Build Tool:** Maven
- **IDE:** IntelliJ IDEA
- **Version Control:** Git and GitHub

System Architecture

The recommended architecture for Sakura Saki follows a layered Spring Boot structure:

- **Entity Layer** - contains the domain classes such as User, Staff, Service, Appointment, Review, and Package.
- **Repository Layer** - contains JPA repository interfaces for database access.
- **Service Layer** - contains business logic and validation.
- **Controller Layer** - handles HTTP requests and connects the backend to the UI.
- **View Layer** - Thymeleaf templates for forms, dashboards, lists, and reports.

This structure helps separate responsibilities clearly and makes it easier for each team member to manage their own module independently.

Team Work Distribution

Member	Module	Main Responsibility
Member 1	Customer & Authentication	Customer registration, login/logout, customer profile management, customer list/search, account update and deactivation.
Member 2	Service & Package Management	Manage salon services and create service packages with category, duration, price, description, and discounts.
Member 3	Staff / Beautician Management	Manage staff registration, staff list/search, profile editing, specialties, and working schedules.
Member 4	Appointment Booking & Schedule	Handle booking creation, viewing schedules, updating appointment status, rescheduling, and conflict checking.
Member 5	Admin Dashboard & Reports	Admin authentication, admin dashboard, admin user management, summary cards, and simple reports.
Member 6	Reviews & Post-Visit Flow	Submit reviews, display reviews, edit/delete own reviews, and admin moderation of inappropriate reviews.

Module Documentation

Member 1 - Customer & Authentication Module

Module Purpose

This module is responsible for managing customer accounts and authentication. It acts as the identity foundation of the whole system because customers must be registered and authenticated before they can book appointments, view profiles, or leave reviews.

Main Features

- Register a new customer account.
- Login and logout using session-based or Spring Security authentication.
- View and update customer profile information.
- Search and list customers.
- Deactivate or delete customer accounts.
- Provide a customer list page for admin-side viewing if required.

Minimum UI Pages

- Customer registration page
- Login page
- Profile page
- Customer update page
- Customer list page (admin view)

CRUD Operations

- **Create:** Register a new customer account.
- **Read:** Search or list customers and view customer profile details.
- **Update:** Edit customer details such as name, email, password, or phone number.
- **Delete:** Deactivate or permanently remove a customer account.

Suggested Classes

- `User.java`
- `Role.java` or role field inside `User`
- `UserRepository.java`
- `UserService.java`
- `AuthController.java`
- `ProfileController.java`
- `SecurityConfig.java`

OOP Concepts Used

- **Encapsulation:** Customer data such as username, password, email, and active status should be private and accessed through getters and setters.
- **Inheritance:** If needed, `AdminUser` and `CustomerUser` can inherit from a common `User` class.
- **Abstraction:** Authentication logic can be separated through service interfaces or security classes.

- **Polymorphism:** Different behaviors can be applied for admin and customer roles when accessing protected pages.

Member 2 - Service & Package Management Module

Module Purpose

This module manages the salon's core service catalog. It stores all services offered by the salon and allows the system to create packages by combining multiple services.

Main Features

- Add new salon services.
- Edit service details such as category, duration, price, and description.
- Delete services that are no longer available.
- Search and list available services.
- Create service packages that combine multiple services.
- Calculate total duration and discounts for packages.

Minimum UI Pages

- Service registration/manage page
- Service list/search page
- Package management page
- Package list page

CRUD Operations

- **Create:** Add a new service or package.
- **Read:** Display services and packages with search/filter support.
- **Update:** Modify service details, package contents, price, duration, or discount.
- **Delete:** Remove a service or package.

Suggested Classes

- `Service.java`
- `Package.java`
- `ServiceRepository.java`
- `PackageRepository.java`
- `ServiceService.java`
- `PackageService.java`
- `ServiceController.java`

- `PackageController.java`

OOP Concepts Used

- **Encapsulation:** Service and package details should be stored in private fields.
- **Inheritance:** Different service types such as `HairService`, `SkinService`, or `BridalService` can extend a common `Service` class if needed.
- **Abstraction:** Common operations such as price calculation or duration calculation can be handled through abstract methods or service-layer interfaces.
- **Polymorphism:** Display or pricing rules may behave differently depending on the service or package type.

Member 3 - Staff / Beautician Management Module

Module Purpose

This module manages all staff or beautician records in the system. It handles staff registration, profile maintenance, specialty assignment, and working day/time availability.

Main Features

- Register new staff members.
- Search or list staff by name or specialty.
- Edit staff profile details.
- Assign specialties or skill tags.
- Maintain working days and available time slots.
- Deactivate or remove staff members.
- Restrict staff management to admin-only access if needed.

Minimum UI Pages

- Staff registration page
- Staff list/search page
- Staff profile page
- Staff edit page

CRUD Operations

- **Create:** Register a new staff member.
- **Read:** Display all staff or search by name and specialty.
- **Update:** Edit staff details including phone, email, specialties, working days, and time slots.
- **Delete:** Deactivate or permanently remove a staff member.

Suggested Classes

- `Staff.java`
- `Stylist.java`
- `Therapist.java`
- `StaffRepository.java`
- `StaffService.java`
- `StaffController.java`

OOP Concepts Used

- **Encapsulation:** All staff fields such as name, email, phone, schedule, and active status remain private with controlled access.
- **Inheritance:** `Stylist` and `Therapist` can extend a common abstract `Staff` class.
- **Abstraction:** `Staff` can be an abstract base class with abstract methods such as `getRole()`.
- **Polymorphism:** The `getRole()` method can return different values depending on whether the object is a `Stylist` or `Therapist`.

Member 4 - Appointment Booking & Schedule Module

Module Purpose

This module manages customer bookings and the day-to-day scheduling logic of the salon. It connects customers, staff, services, and time slots into one appointment workflow.

Main Features

- Create a new appointment booking.
- Link a booking to a customer, a service or package, a staff member, and a date/time.
- View upcoming appointments by customer, staff, or date.
- Update booking details.
- Reschedule appointments.
- Change appointment status such as Booked, Completed, or Cancelled.
- Delete or cancel appointments.
- Prevent double booking for the same staff member within the same time range.

Minimum UI Pages

- Booking form page
- My appointments page
- Staff appointments page
- Appointment update/reschedule page

CRUD Operations

- **Create:** Create a new appointment booking.
- **Read:** View appointments by customer, staff member, or date.
- **Update:** Reschedule appointment, update service/staff assignment, or change appointment status.
- **Delete:** Cancel or remove an appointment record.

Suggested Classes

- Appointment.java
- AppointmentRepository.java
- AppointmentService.java
- AppointmentController.java
- ScheduleValidator.java or schedule validation methods inside the service layer

OOP Concepts Used

- **Encapsulation:** Appointment fields such as customer, staff, service, date, time, and status should be private.
- **Abstraction:** Conflict-checking and scheduling rules can be abstracted into dedicated service methods.
- **Inheritance:** If the team decides to create multiple booking types, they can inherit from a common base appointment class.
- **Polymorphism:** Different update or display behavior can be applied depending on appointment status or appointment type.

Member 5 - Admin Dashboard & Reports Module

Module Purpose

This module manages administrative access and overall system monitoring. It provides the admin dashboard, admin user control, system summaries, and report generation.

Main Features

- Admin login authentication.
- Protect admin routes so only admin users can access them.
- Display dashboard cards such as total customers, total services, total staff, and today's appointments.
- Provide a simple reports page.
- Generate reports such as appointments per day and most booked services.
- Manage admin users through CRUD operations.

- Search, filter, and paginate admin or user records.
- Provide logout and protected navigation.

Minimum UI Pages

- Admin login page
- Admin dashboard page
- Reports page
- Admin management page
- User management page

CRUD Operations

- **Create:** Add a new admin user if the design supports admin creation.
- **Read:** View dashboard statistics, reports, and lists of admins or users.
- **Update:** Update admin details, role assignments, and user activation states.
- **Delete:** Remove admin users or deactivate managed user accounts where allowed.

Suggested Classes

- `AdminController.java`
- `AdminService.java`
- `ReportService.java`
- `DashboardSummaryDTO.java`
- `AdminRepository.java` or `shared UserRepository.java`

OOP Concepts Used

- **Encapsulation:** Admin data and dashboard summary data should be protected inside classes.
- **Inheritance:** Admin accounts can inherit from a shared user model.
- **Abstraction:** Reporting logic and admin-only operations can be separated into service-layer abstractions.
- **Polymorphism:** Role-based behavior differs between normal users and administrators when performing actions.

Member 6 - Reviews & Post-Visit Flow Module

Module Purpose

This module manages customer feedback after appointments. It allows customers to review services or staff and provides moderation features for the admin side.

Main Features

- Submit a rating and comment after an appointment.
- View reviews for a service.
- View reviews for a staff member.
- Edit or delete one's own review.
- Allow admins to hide or remove inappropriate reviews.
- Optionally restrict reviews to completed appointments only.

Minimum UI Pages

- Submit review page
- View reviews page
- Review edit page
- Admin moderation page

CRUD Operations

- **Create:** Submit a new review.
- **Read:** View all reviews for a service or staff member.
- **Update:** Edit the review content or rating, or update moderation status.
- **Delete:** Delete a review by the author or by admin moderation.

Suggested Classes

- `Review.java`
- `ReviewRepository.java`
- `ReviewService.java`
- `ReviewController.java`
- `ModerationService.java`

OOP Concepts Used

- **Encapsulation:** Review data such as rating, comment, author, and visibility status should remain private.
- **Inheritance:** Different review types can be introduced if needed, such as `VerifiedReview` and `PublicReview`.

- **Abstraction:** Moderation and visibility rules can be handled through abstract service behavior.
- **Polymorphism:** Customer and admin may interact with the same review object differently depending on role.

Common CRUD Summary for the Whole System

Module	Create	Read	Update	Delete
Customer & Authentication	Register customer	Search/list/view profile	Edit profile or customer details	Deactivate/delete customer
Service & Package	Add service/package	View/search services and packages	Edit service/package	Delete service/package
Staff Management	Add staff	List/search staff	Edit staff details/schedule	Deactivate/delete staff
Appointments	Create booking	View schedules/bookings	Reschedule/update status	Cancel/delete booking
Admin Dashboard	Create admin	View dashboard/reports/admins	Update permissions/details	Remove admin/deactivate users
Reviews	Add review	View reviews	Edit review/moderation status	Delete/hide review

Common OOP Concepts Across the Project

1. Encapsulation

Every main entity should store its fields as private and expose them through getters, setters, and service-layer methods. This protects data from direct unsafe modification.

2. Inheritance

Inheritance can be used to reduce duplication when multiple classes share common fields. Example structures include:

- User -> AdminUser, CustomerUser
- Staff -> Stylist, Therapist
- Service -> different service categories if needed

3. Abstraction

Abstract classes and interfaces help define common behavior while hiding implementation details. Good examples include:

- Staff as an abstract class

- service-layer interfaces such as `AppointmentService`
- reporting methods that hide internal query logic

4. Polymorphism

Polymorphism allows the same method name to behave differently depending on the object or role involved. Example cases include:

- `getRole()` returning different results based on staff subtype
- different dashboard or page access behavior for admin and customer users
- different display methods for review visibility or service types

Suggested Database Entities

The following entities are likely to be needed in the final integrated system:

- User
- Role
- Service
- Package
- Staff
- Appointment
- Review
- Admin activity or audit log (optional)

Suggested Page Flow

- Home page
- Register / Login
- Customer profile
- Service list and package list
- Staff list
- Appointment booking page
- Review submission page
- Admin login
- Admin dashboard
- Reports page
- Admin management pages

Integration Between Modules

The modules are separate for workload distribution, but they are functionally connected:

- Customers must be registered before booking appointments.
- Services and staff must exist before an appointment can be created.
- Reviews should ideally be linked to completed appointments.
- Admin dashboard summaries depend on data from customer, service, staff, and appointment modules.

Scope Boundaries

Each member should focus primarily on the assigned module unless the team agrees otherwise. This helps keep the Git history, viva explanation, and code ownership clear.

Examples:

- Member 3 should not take over full booking logic.
- Member 5 should not own the service management UI unless reassigned.
- Member 6 should not redesign authentication unless it directly affects review permissions.

Deliverables for Final Submission

The final project submission should ideally include:

- Full source code
- GitHub repository link and commit history
- Class diagrams
- Final report
- Sample data for demonstration
- Minimum three working UIs per member module
- Demonstration of CRUD features
- Explanation of OOP concepts in backend code

Conclusion

Sakura Saki is structured as a six-module OOP-based salon management system where each member handles one clear responsibility area. The combined system demonstrates CRUD operations, layered Spring Boot architecture, role-based access, and practical use of Object-Oriented Programming principles. This documentation can be used as the team's final work distribution and module explanation document, with small edits later if the scope changes before final submission.